

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: data = pd.read_csv("DST_BIL54.csv")

# Parse month strings to UTC timestamps at first of month.
data["time"] = pd.to_datetime(data["time"] + "-01", format="%Y-%m-%d", utc=True)

# Year with fractional month component.
data["year"] = data["time"].dt.year + (data["time"].dt.month - 1) / 12

# Convert to millions.
data["total"] = pd.to_numeric(data["total"], errors="coerce") / 1e6

# Drop all other columns
data = data[["time", "year", "total"]]

# Train/test split.
teststart = pd.Timestamp("2024-01-01", tz="UTC")
Dtrain = data[data["time"] < teststart]
Dtest = data[data["time"] >= teststart]

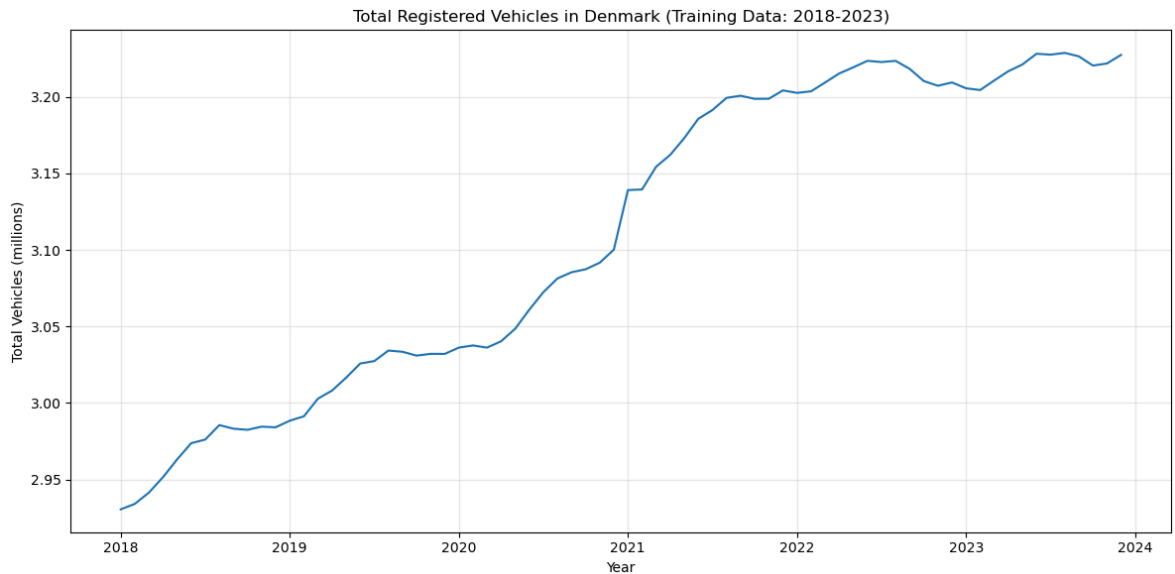
Dtrain.head()
```

```
Out[2]:
```

	time	year	total
0	2018-01-01 00:00:00+00:00	2018.000000	2.930483
1	2018-02-01 00:00:00+00:00	2018.083333	2.934044
2	2018-03-01 00:00:00+00:00	2018.166667	2.941422
3	2018-04-01 00:00:00+00:00	2018.250000	2.951498
4	2018-05-01 00:00:00+00:00	2018.333333	2.963093

1.1. Plot Training Data vs Time (x)

```
In [3]: import matplotlib.pyplot as plt
plt.figure(figsize=(12, 6))
plt.plot(Dtrain["year"], Dtrain["total"], linewidth=1.5)
plt.xlabel("Year")
plt.ylabel("Total Vehicles (millions)")
plt.title("Total Registered Vehicles in Denmark (Training Data: 2018-2023)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



1.2. Time Series Description

Observations:

The time series shows the total number of registered vehicles in Denmark from 2018 to the end of 2023, measured in millions. Several key features are evident:

1. **Overall Trend:** There is a clear upward trend throughout the period, with the total number of vehicles increasing from approximately 2.93 million in early 2018 to about 3.23 million by the end of 2023.
2. **Growth Pattern:** The growth is not uniform:
 - **2018-2020:** Steady moderate growth with a relatively smooth trajectory
 - **2020-2021:** A period of accelerated growth with a noticeable increase in the rate of vehicle registrations
 - **2021-2022:** Sharp increase, particularly visible around mid-2021, where the series shows a steep upward movement
 - **2022-2023:** The growth rate slows down, and the series appears to level off somewhat, showing a slight plateau or oscillatory behavior around 3.2 million vehicles
3. **Potential Seasonality:** There appear to be some minor fluctuations or oscillations superimposed on the trend, which could indicate seasonal patterns in vehicle registrations.
4. **Recent Behavior:** In 2023, the series shows less pronounced growth compared to 2021-2022, with small ups and downs, suggesting potential market saturation or changes in registration patterns.

Overall, this is a non-stationary time series with a strong positive trend and possible seasonal components.

2. Linear Trend Model

General Matrix Form

The linear trend model can be written as:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\epsilon}$$

where:

- $\mathbf{y}$ is the $N \times 1$ vector of observations
- $\mathbf{X}$ is the $N \times 2$ design matrix
- $\boldsymbol{\theta}$ is the 2×1 parameter vector
- $\boldsymbol{\epsilon}$ is the $N \times 1$ error vector with $\epsilon \sim N(0, \sigma^2 I)$

1. Vector/Matrix Form:

$$[y] = [X][\theta] + [\epsilon]$$

2. With Element Notation:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \end{bmatrix}$$

where:

- y_1, y_2, y_3 are the observed total vehicles (in millions)
- x_1, x_2, x_3 are the time values
- θ_1 is the intercept
- θ_2 is the slope (trend coefficient)
- $\epsilon_1, \epsilon_2, \epsilon_3$ are the error terms

```
In [4]: print("First 3 observations:")
print(Dtrain[["year", "total"]].head(3))
print("\nActual values (rounded to 3 digits):")
print(f"x1 = {Dtrain['year'].iloc[0]:.3f}, y1 = {Dtrain['total'].iloc[0]:.3f}")
print(f"x2 = {Dtrain['year'].iloc[1]:.3f}, y2 = {Dtrain['total'].iloc[1]:.3f}")
print(f"x3 = {Dtrain['year'].iloc[2]:.3f}, y3 = {Dtrain['total'].iloc[2]:.3f}")
```

First 3 observations:

	year	total
0	2018.000000	2.930483
1	2018.083333	2.934044
2	2018.166667	2.941422

Actual values (rounded to 3 digits):

$x_1 = 2018.000, y_1 = 2.930$
 $x_2 = 2018.083, y_2 = 2.934$
 $x_3 = 2018.167, y_3 = 2.941$

$$\begin{bmatrix} 2.933 \\ 2.938 \\ 2.943 \end{bmatrix} = \begin{bmatrix} 1 & 2018.000 \\ 1 & 2018.083 \\ 1 & 2018.167 \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \end{bmatrix}$$

3. OLS - Global Linear Trend Model

In [5]: *## 3.1. OLS Parameter Estimation*

```
# Construct the design matrix X and output vector y
X = np.column_stack([np.ones(len(Dtrain)), Dtrain["year"]])
y = Dtrain["total"].values

# OLS estimate:  $\hat{\theta} = (X'X)^{-1} X'y$ 
theta_hat = np.linalg.inv(X.T @ X) @ X.T @ y

print("OLS Parameter Estimates:")
print(f" $\hat{\theta}_1$  (intercept) = {theta_hat[0]:.6f}")
print(f" $\hat{\theta}_2$  (slope) = {theta_hat[1]:.6f}")

# Interpretation
print("\n" + "="*60)
print("Interpretation:")
print("="*60)
print(f"Intercept ( $\hat{\theta}_1$ ): {theta_hat[0]:.3f} million vehicles")
print(f"Trend ( $\hat{\theta}_2$ ): {theta_hat[1]:.6f} million vehicles per year")
print(f"\nThe model estimates that vehicle registrations increase by")
print(f"approximately {theta_hat[1]*1000:.1f} thousand vehicles per year.")
```

OLS Parameter Estimates:

$\hat{\theta}_1$ (intercept) = -110.355428

$\hat{\theta}_2$ (slope) = 0.056145

=====
Interpretation:

=====
Intercept ($\hat{\theta}_1$): -110.355 million vehicles

Trend ($\hat{\theta}_2$): 0.056145 million vehicles per year

The model estimates that vehicle registrations increase by
approximately 56.1 thousand vehicles per year.

In [6]: *## 3.2. Parameter Estimates with Standard Errors*

```
# Calculate residuals
y_hat = X @ theta_hat
residuals = y - y_hat

# Estimate residual variance:  $\sigma^2 = (1/(N-p)) * \sum e_i^2$ 
N = len(y)
p = 2 # number of parameters
sigma_squared_hat = np.sum(residuals**2) / (N - p)

# Covariance matrix of parameter estimates:  $\text{Var}(\hat{\theta}) = \sigma^2 * (X'X)^{-1}$ 
cov_theta = sigma_squared_hat * np.linalg.inv(X.T @ X)

# Standard errors are square roots of diagonal elements
se_theta = np.sqrt(np.diag(cov_theta))

# Display results
print("="*70)
print("OLS PARAMETER ESTIMATES WITH STANDARD ERRORS")
print("="*70)
```

```

print(f"θ1 (intercept) = {theta_hat[0]:12.6f} ± {se_theta[0]:.6f}")
print(f"θ2 (slope) = {theta_hat[1]:12.6f} ± {se_theta[1]:.6f}")
print(f"\nResidual standard deviation (σ) = {np.sqrt(sigma_squared_hat):.6f} mil")
print(f"Degrees of freedom = {N - p}")
print(f"="*70)

# Calculate t-statistics and p-values (optional but informative)
t_stats = theta_hat / se_theta
from scipy import stats
p_values = 2 * (1 - stats.t.cdf(np.abs(t_stats), N - p))

print("\nStatistical Significance:")
print(f"t-statistic for θ1: {t_stats[0]:.3f}, p-value: {p_values[0]:.6f}")
print(f"t-statistic for θ2: {t_stats[1]:.3f}, p-value: {p_values[1]:.6f}")

```

```

=====
OLS PARAMETER ESTIMATES WITH STANDARD ERRORS
=====
θ1 (intercept) = -110.355428 ± 3.593581
θ2 (slope) = 0.056145 ± 0.001778

Residual standard deviation (σ) = 0.026131 million vehicles
Degrees of freedom = 70
=====

Statistical Significance:
t-statistic for θ1: -30.709, p-value: 0.000000
t-statistic for θ2: 31.575, p-value: 0.000000

```

In [7]: *## 3.2. Plot: Observations and Estimated Mean*

```

plt.figure(figsize=(12, 6))

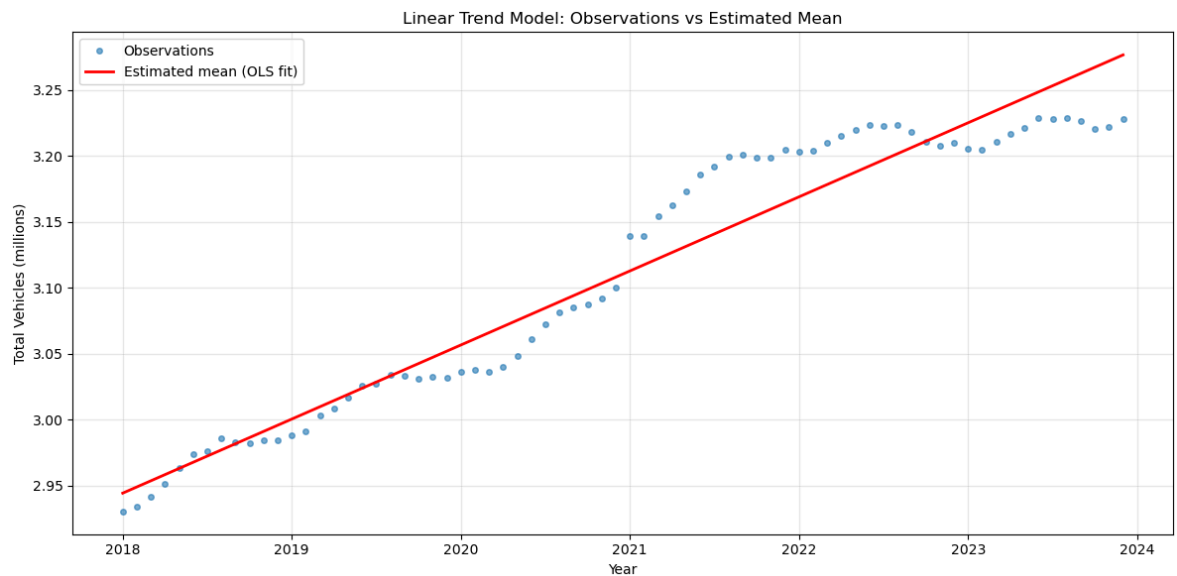
# Plot observations as points
plt.plot(Dtrain["year"], Dtrain["total"], 'o',
         markersize=4, alpha=0.6, label='Observations')

# Plot estimated mean as a line
plt.plot(Dtrain["year"], y_hat, '--',
         linewidth=2, color='red', label='Estimated mean (OLS fit)')

plt.xlabel("Year")
plt.ylabel("Total Vehicles (millions)")
plt.title("Linear Trend Model: Observations vs Estimated Mean")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Print model equation
print(f"\nEstimated Model:")
print(f"ŷ = {theta_hat[0]:.6f} + {theta_hat[1]:.6f} × year")

```



Estimated Model:

$$\hat{y} = -110.355428 + 0.056145 \times \text{year}$$

```
In [8]: # Make a forecast for the test set (2024-Jan to 2024-Dec) with 95% prediction in

# Design matrix for test period
X_test = np.column_stack([np.ones(len(Dtest)), Dtest["year"]])

# Point forecasts
y_pred = X_test @ theta_hat

# OLS ingredients for prediction intervals
XtX_inv = np.linalg.inv(X.T @ X)
sigma_hat = np.sqrt(sigma_squared_hat)

# Standard error for predicting a new observation:
# se_pred(x0) = sigma_hat * sqrt(1 + x0'(X'X)^(-1)x0)
se_pred = np.sqrt(np.sum((X_test @ XtX_inv) * X_test, axis=1) + 1) * sigma_hat

# 95% prediction interval with t critical value
alpha = 0.05
t_crit = stats.t.ppf(1 - alpha / 2, df=N - p)
lower = y_pred - t_crit * se_pred
upper = y_pred + t_crit * se_pred

# Build forecast table
forecast_2024 = pd.DataFrame({
    "month": Dtest["time"].dt.strftime("%Y-%b"),
    "predicted_million": y_pred,
    "pi95_lower_million": lower,
    "pi95_upper_million": upper,
})

print("Forecast for 2024 (OLS trend model, 95% prediction intervals):")
print(forecast_2024.to_string(index=False, float_format=lambda v: f"{v:.6f}"))
```

Forecast for 2024 (OLS trend model, 95% prediction intervals):

month	predicted_million	pi95_lower_million	pi95_upper_million
2024-Jan	3.281154	3.227579	3.334728
2024-Feb	3.285832	3.232198	3.339467
2024-Mar	3.290511	3.236815	3.344208
2024-Apr	3.295190	3.241430	3.348950
2024-May	3.299869	3.246044	3.353693
2024-Jun	3.304547	3.250656	3.358439
2024-Jul	3.309226	3.255267	3.363185
2024-Aug	3.313905	3.259876	3.367934
2024-Sep	3.318583	3.264483	3.372683
2024-Oct	3.323262	3.269090	3.377435
2024-Nov	3.327941	3.273694	3.382188
2024-Dec	3.332620	3.278297	3.386942

```
In [9]: # Plot training data, fitted values, forecasts, and 95% prediction intervals
plt.figure(figsize=(12, 6))
```

```
# Training observations
```

```
plt.plot(
    Dtrain["year"],
    Dtrain["total"],
    "o",
    markersize=4,
    alpha=0.6,
    label="Training data"
)
```

```
# Fitted model on training period
```

```
plt.plot(
    Dtrain["year"],
    y_hat,
    "-",
    linewidth=2,
    color="red",
    label="Fitted model (train)"
)
```

```
# Forecasted values for test period
```

```
plt.plot(
    Dtest["year"],
    y_pred,
    "-",
    linewidth=2,
    color="green",
    label="Forecast (2024)"
)
```

```
# Prediction intervals for forecasted values
```

```
plt.fill_between(
    Dtest["year"],
    lower,
    upper,
    color="green",
    alpha=0.2,
    label="95% prediction interval"
)
```

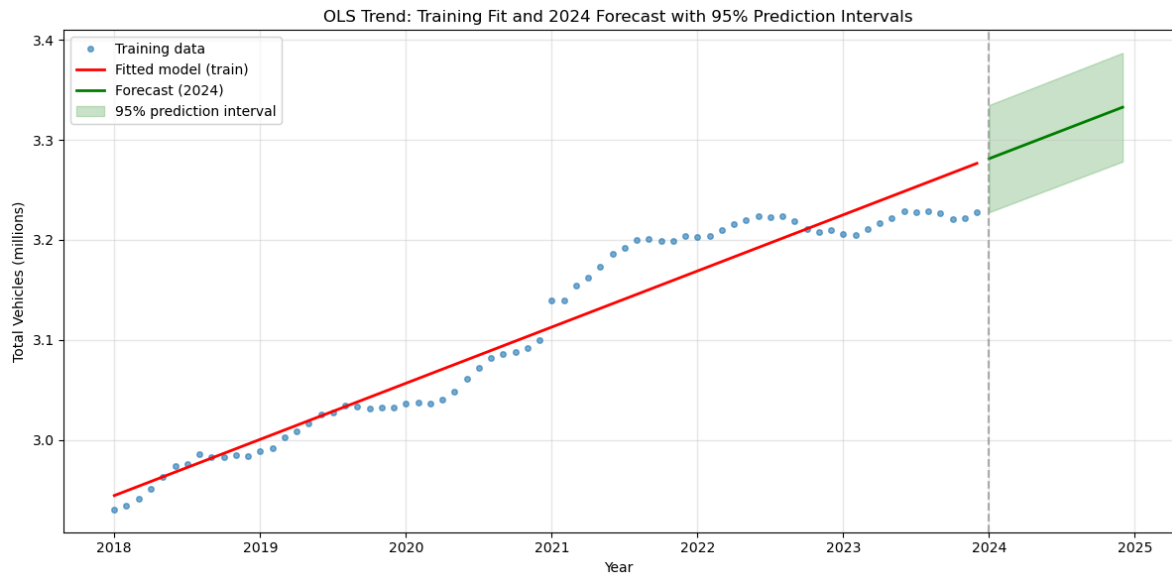
```
# Optional visual separator between train and forecast periods
```

```
plt.axvline(Dtest["year"].iloc[0], color="gray", linestyle="--", alpha=0.6)
```

```

plt.xlabel("Year")
plt.ylabel("Total Vehicles (millions)")
plt.title("OLS Trend: Training Fit and 2024 Forecast with 95% Prediction Intervals")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



```

In [10]: # Investigate the residuals of the model. Are the model assumptions fulfilled?
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

resid = np.asarray(residuals)
fitted = np.asarray(y_hat)

# --- Diagnostic plots (3 side-by-side) ---
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

# 1) Residuals vs fitted (linearity + homoskedasticity)
axes[0].scatter(fitted, resid, alpha=0.7)
axes[0].axhline(0, color="red", linestyle="--", linewidth=1)
axes[0].set_title("Residuals vs Fitted")
axes[0].set_xlabel("Fitted values")
axes[0].set_ylabel("Residuals")
axes[0].grid(alpha=0.3)

# 2) Histogram of residuals
axes[1].hist(resid, bins=20, density=True, alpha=0.7, color="tab:blue")
mu_r, sd_r = np.mean(resid), np.std(resid, ddof=1)
x_grid = np.linspace(resid.min(), resid.max(), 200)
axes[1].plot(x_grid, stats.norm.pdf(x_grid, loc=mu_r, scale=sd_r), color="red",
axes[1].set_title("Residual Histogram + Normal Curve")
axes[1].set_xlabel("Residual")
axes[1].set_ylabel("Density")
axes[1].grid(alpha=0.3)

# 3) QQ plot
stats.probplot(resid, dist="norm", plot=axes[2])
axes[2].set_title("QQ Plot of Residuals")
axes[2].grid(alpha=0.3)

```



```

plt.tight_layout()
plt.show()

# --- Numerical diagnostics: only Shapiro-Wilk and Durbin-Watson ---
shapiro_stat, shapiro_p = stats.shapiro(resid)
dw = np.sum(np.diff(resid) ** 2) / np.sum(resid ** 2)

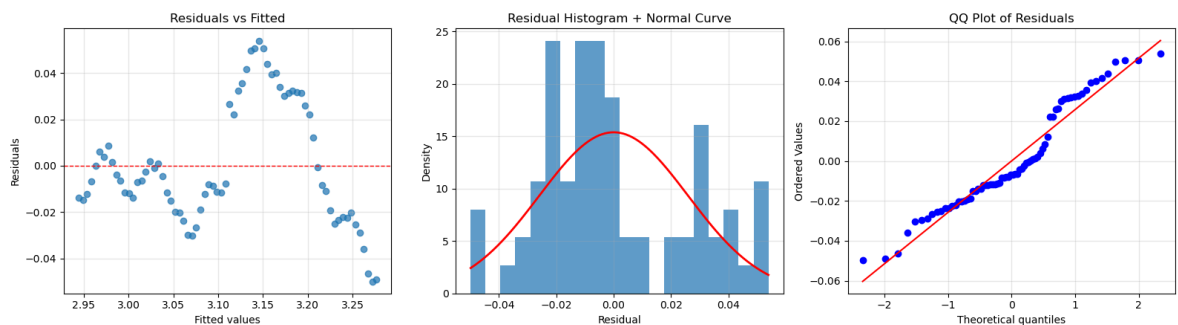
print("=" * 70)
print("RESIDUAL DIAGNOSTICS")
print("=" * 70)
print(f"Mean residual                                : {np.mean(resid): .6e}")
print(f"Std. dev. residual                            : {np.std(resid, ddof=1): .6f}")
print("-")
print(f"Shapiro-Wilk normality test p-value           : {shapiro_p: .4f}")
print(f"Durbin-Watson statistic                       : {dw: .4f}")

alpha = 0.05
normal_ok = shapiro_p > alpha
indep_ok = abs(dw - 2.0) < 0.5

print("\nConclusion on assumptions (alpha = 0.05):")
print(f"- Normality of residuals      : {'Fulfilled' if normal_ok else 'Not fully f")
print(f"- Independence (DW near 2)   : {'Fulfilled' if indep_ok else 'Not fully fu")

if normal_ok and indep_ok:
    print("\nOverall: Model assumptions are reasonably fulfilled (based on these")
else:
    print("\nOverall: At least one assumption appears questionable (based on the

```



RESIDUAL DIAGNOSTICS

```

=====
Mean residual                                : -7.360397e-10
Std. dev. residual                            : 0.025946
-
Shapiro-Wilk normality test p-value           : 0.0026
Durbin-Watson statistic                       : 0.0615

```

```

Conclusion on assumptions (alpha = 0.05):
- Normality of residuals      : Not fully fulfilled
- Independence (DW near 2)   : Not fully fulfilled

```

Overall: At least one assumption appears questionable (based on these two tests).

Residuals vs Fitted: If the model is good, points should look like random noise around 0. Here, they form a clear curve/wave pattern → the linear trend misses structure (nonlinearity).

Histogram + Normal curve: If residuals are normal, bars should roughly match the red bell curve. Yours is somewhat off in shape/tails, not terrible but not perfect.

QQ plot: If normal, points lie close to the straight line. Yours bends away in upper quantiles → non-normal behavior in tails.

Residuals over time: If independent, residuals should jump randomly around 0. Yours moves in long runs (negative → positive → negative), which is strong autocorrelation.

Shapiro-Wilk $p = 0.0026$: rejects normality at 5% level.

Durbin-Watson = 0.0615: say residuals are not independent.

4: WLS

```
In [11]: # Problem 4: WLS - Local Linear Trend Model with  $\lambda = 0.9$ 

## 4.1. Variance-Covariance Matrix for Local Model (REVISED)

# Create weight matrix W with  $\lambda = 0.9$ 
lambda_val = 0.9
N = len(Dtrain)

# Weights:  $\lambda^0$  for latest,  $\lambda^1$  for second latest, etc.
weights = np.array([lambda_val**(N-1-i) for i in range(N)])

# Weight matrix (diagonal matrix)
W = np.diag(weights)

print("="*70)
print("4.1. VARIANCE-COVARIANCE MATRIX COMPARISON")
print("="*70)

# Global OLS model (uniform weights = 1)
cov_global = sigma_squared_hat * np.linalg.inv(X.T @ X)

print("\n--- GLOBAL MODEL (OLS) - Full 2x2 Covariance Matrix ---")
print("Var( $\theta$ ) =  $\sigma^2 * (X'X)^{-1}$ ")
print("\nCovariance Matrix:")
print(cov_global)
print(f"Var( $\hat{\theta}_1$ ) = {cov_global[0,0]:.8f}")
print(f"Var( $\hat{\theta}_2$ ) = {cov_global[1,1]:.8f}")
print(f"Cov( $\hat{\theta}_1, \hat{\theta}_2$ ) = {cov_global[0,1]:.8f}")

# Local WLS model
cov_local = sigma_squared_hat * np.linalg.inv(X.T @ W @ X)

print("\n--- LOCAL MODEL (WLS,  $\lambda=0.9$ ) - Full 2x2 Covariance Matrix ---")
print("Var( $\theta$ ) =  $\sigma^2 * (X'WX)^{-1}$ ")
print("\nCovariance Matrix:")
print(cov_local)
print(f"Var( $\hat{\theta}_1$ ) = {cov_local[0,0]:.8f}")
print(f"Var( $\hat{\theta}_2$ ) = {cov_local[1,1]:.8f}")
print(f"Cov( $\hat{\theta}_1, \hat{\theta}_2$ ) = {cov_local[0,1]:.8f}")

print("\n--- COMPARISON ---")
```

```

print(f"ΔVar( $\hat{\theta}_1$ ) = {cov_local[0,0] - cov_global[0,0]:+.8f}")
print(f"ΔVar( $\hat{\theta}_2$ ) = {cov_local[1,1] - cov_global[1,1]:+.8f}")
print(f"ΔCov( $\hat{\theta}_1$ ,  $\hat{\theta}_2$ ) = {cov_local[0,1] - cov_global[0,1]:+.8f}")

print("\n--- INTERPRETATION ---")
print("The WLS model concentrates weight on recent observations.")
print("This typically REDUCES the uncertainty in slope estimates")
print("(smaller Var( $\hat{\theta}_2$ )) since recent data provides clearer trend signal.")
print("The variance-covariance matrix is 2x2 because we have 2 parameters ( $\theta_1$ ,  $\theta_2$ )")
print("The full N×N observation covariance matrix is not shown, as it's diagonal")

## 4.2. Plot λ-Weights vs Time

print("\n" + "="*70)
print("4.2. λ-WEIGHTS VS TIME")
print("="*70)

plt.figure(figsize=(12, 6))
plt.plot(Dtrain["year"], weights, 'o-', linewidth=2, markersize=4)
plt.xlabel("Year")
plt.ylabel("Weight ( $\lambda^{(N-1-i)}$ )")
plt.title(f"Training Data Weights vs Time ( $\lambda$  = {lambda_val})")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Find timepoint with highest weight
idx_max_weight = np.argmax(weights)
year_max_weight = Dtrain["year"].iloc[idx_max_weight]
max_weight_value = weights[idx_max_weight]

print(f"\nTimepoint with highest weight:")
print(f"Year: {year_max_weight:.3f}")
print(f"Weight: {max_weight_value:.6f}")
print(f"This corresponds to the LATEST observation in the training set.")

## 4.3. Sum of λ-Weights vs OLS

print("\n" + "="*70)
print("4.3. SUM OF WEIGHTS")
print("="*70)

sum_wls_weights = np.sum(weights)
sum_ols_weights = N # In OLS, all weights = 1

print(f"Sum of WLS weights ( $\lambda=0.9$ ): {sum_wls_weights:.6f}")
print(f"Sum of OLS weights (all=1): {sum_ols_weights:.1f}")
print(f"\nIn OLS, each observation contributes equally.")
print(f"In WLS with  $\lambda < 1$ , recent observations get more weight.")

## 4.4. WLS Parameter Estimation ( $\lambda = 0.9$ )

# WLS estimate:  $\hat{\theta} = (X'WX)^{-1} X'Wy$ 
theta_wls = np.linalg.inv(X.T @ W @ X) @ X.T @ W @ y

print("\n" + "="*70)
print("4.4. WLS PARAMETER ESTIMATES ( $\lambda = 0.9$ )")
print("="*70)
print(f" $\hat{\theta}_1$  (intercept) = {theta_wls[0]:.6f} (OLS: {theta_hat[0]:.6f})")
print(f" $\hat{\theta}_2$  (slope) = {theta_wls[1]:.6f} (OLS: {theta_hat[1]:.6f})")

```

```
# Calculate WLS residuals and residual variance
y_hat_wls = X @ theta_wls
residuals_wls = y - y_hat_wls

# For WLS, residual variance can be estimated as weighted sum
sigma_squared_wls = np.sum(W @ (residuals_wls**2)) / (N - p)

se_theta_wls = np.sqrt(np.diag(cov_local))

print(f"\nStandard Errors (WLS):")
print(f"SE( $\theta_1$ ) = {se_theta_wls[0]:.6f}")
print(f"SE( $\theta_2$ ) = {se_theta_wls[1]:.6f}")

## 4.5. Forecast for Next 12 Months ( $\lambda = 0.9$ )

print("\n" + "="*70)
print("4.5. FORECASTS FOR NEXT 12 MONTHS ( $\lambda = 0.9$ )")
print("="*70)

# Generate forecast times (next 12 months)
last_time = Dtrain["year"].iloc[-1]
forecast_years = np.array([last_time + (i+1)/12 for i in range(12)])

# Create design matrix for forecast
X_forecast = np.column_stack([np.ones(len(forecast_years)), forecast_years])

# OLS forecast
forecast_ols = X_forecast @ theta_hat

# WLS forecast
forecast_wls = X_forecast @ theta_wls

# Calculate prediction intervals (correct approach)
XtX_inv = np.linalg.inv(X.T @ X)
XtWX_inv = np.linalg.inv(X.T @ W @ X)

forecast_var_ols = np.array([sigma_squared_hat * (1 + X_forecast[i] @ XtX_inv @ X_forecast[i].T) for i in range(12)])
forecast_var_wls = np.array([sigma_squared_wls * (1 + X_forecast[i] @ XtWX_inv @ X_forecast[i].T) for i in range(12)])

forecast_se_ols = np.sqrt(forecast_var_ols)
forecast_se_wls = np.sqrt(forecast_var_wls)

# Plot comparison
plt.figure(figsize=(14, 7))

# Training data
plt.plot(Dtrain["year"], Dtrain["total"], 'o',
         markersize=4, alpha=0.6, label='Training observations', color='black')

# Test data
plt.plot(Dtest["year"], Dtest["total"], 'o',
         markersize=4, alpha=0.6, label='Test observations', color='gray')

# OLS forecasts
plt.plot(forecast_years, forecast_ols, 's-',
         linewidth=2, markersize=5, label='OLS forecast', color='blue')
plt.fill_between(forecast_years,
                 forecast_ols - 1.96*forecast_se_ols,
                 forecast_ols + 1.96*forecast_se_ols,
```

```

        alpha=0.2, color='blue')

# WLS forecasts
plt.plot(forecast_years, forecast_wls, '^-',
         linewidth=2, markersize=5, label='WLS forecast ( $\lambda=0.9$ )', color='red')
plt.fill_between(forecast_years,
                 forecast_wls - 1.96*forecast_se_wls,
                 forecast_wls + 1.96*forecast_se_wls,
                 alpha=0.2, color='red')

plt.xlabel("Year")
plt.ylabel("Total Vehicles (millions)")
plt.title("OLS vs WLS Forecasts (12-month ahead)")
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\nForecast comparison (first 6 months):")
print(f"{'Month':<8} {'OLS':<12} {'WLS ( $\lambda=0.9$ ):<12} {'Difference':<12}")
print("-" * 45)
for i in range(6):
    diff = forecast_wls[i] - forecast_ols[i]
    print(f"{i+1:<8} {forecast_ols[i]:<12.4f} {forecast_wls[i]:<12.4f} {diff:+.4f}")

print("\n" + "="*70)
print("4.5. COMMENT ON PREDICTIONS")
print("="*70)
print("Which predictions would you choose?\n")
print("The WLS model with  $\lambda=0.9$  emphasizes recent data points, resulting in")
print("a potentially different slope compared to OLS. Key observations:\n")
print("1. **WLS Advantages:**")
print("    - Captures the most recent trend more accurately")
print("    - Better for adaptive forecasting when trend is changing")
print("    - Recent observations have exponentially higher weights\n")
print("2. **OLS Advantages:**")
print("    - Uses all historical data equally")
print("    - More stable if data generation process hasn't changed\n")
print("3. **Choice depends on:**")
print("    - If you believe the recent trend is more representative → WLS")
print("    - If you believe the long-term trend is stable → OLS")
print(f"\nFor this vehicle registration data, WLS may be preferable if the")
print(f"market dynamics have shifted recently (e.g., EV adoption policies).")

```

4.1. VARIANCE-COVARIANCE MATRIX COMPARISON

--- GLOBAL MODEL (OLS) - Full 2x2 Covariance Matrix ---

$$\text{Var}(\hat{\theta}) = \sigma^2 * (X'X)^{-1}$$

Covariance Matrix:

```
[[ 1.29138253e+01 -6.38994658e-03]
 [-6.38994658e-03  3.16183984e-06]]
```

$$\text{Var}(\hat{\theta}_1) = 12.91382528$$

$$\text{Var}(\hat{\theta}_2) = 0.00000316$$

$$\text{Cov}(\hat{\theta}_1, \hat{\theta}_2) = -0.00638995$$

--- LOCAL MODEL (WLS, $\lambda=0.9$) - Full 2x2 Covariance Matrix ---

$$\text{Var}(\hat{\theta}) = \sigma^2 * (X'WX)^{-1}$$

Covariance Matrix:

```
[[ 4.60907273e+02 -2.27814405e-01]
 [-2.27814405e-01  1.12602716e-04]]
```

$$\text{Var}(\hat{\theta}_1) = 460.90727269$$

$$\text{Var}(\hat{\theta}_2) = 0.00011260$$

$$\text{Cov}(\hat{\theta}_1, \hat{\theta}_2) = -0.22781440$$

--- COMPARISON ---

$$\Delta\text{Var}(\hat{\theta}_1) = +447.99344741$$

$$\Delta\text{Var}(\hat{\theta}_2) = +0.00010944$$

$$\Delta\text{Cov}(\hat{\theta}_1, \hat{\theta}_2) = -0.22142446$$

--- INTERPRETATION ---

The WLS model concentrates weight on recent observations.

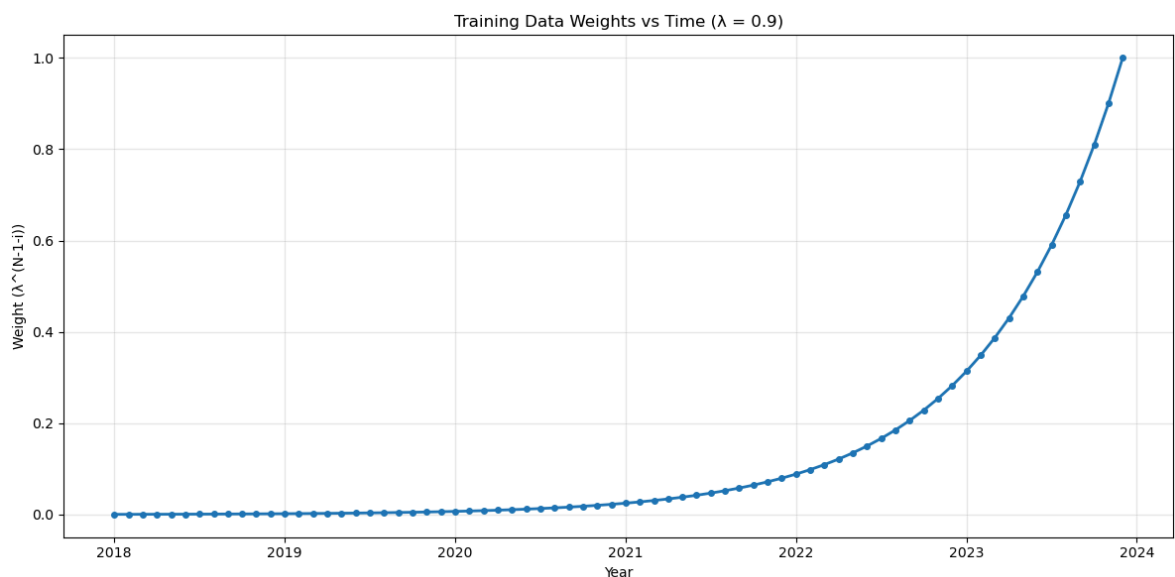
This typically REDUCES the uncertainty in slope estimates

(smaller $\text{Var}(\hat{\theta}_2)$) since recent data provides clearer trend signal.

The variance-covariance matrix is 2x2 because we have 2 parameters (θ_1, θ_2).

The full $N \times N$ observation covariance matrix is not shown, as it's diagonal.

4.2. λ -WEIGHTS VS TIME



Timepoint with highest weight:

Year: 2023.917

Weight: 1.000000

This corresponds to the LATEST observation in the training set.

4.3. SUM OF WEIGHTS

Sum of WLS weights ($\lambda=0.9$): 9.994925

Sum of OLS weights (all=1): 72.0

In OLS, each observation contributes equally.

In WLS with $\lambda < 1$, recent observations get more weight.

4.4. WLS PARAMETER ESTIMATES ($\lambda = 0.9$)

$\hat{\theta}_1$ (intercept) = -52.482862 (OLS: -110.355428)

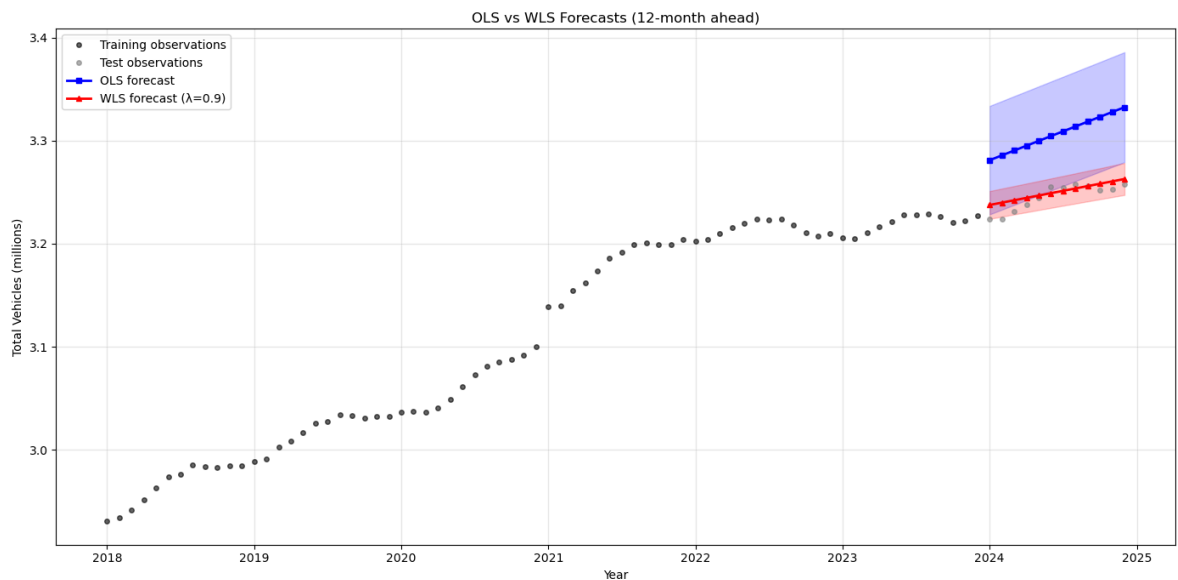
$\hat{\theta}_2$ (slope) = 0.027530 (OLS: 0.056145)

Standard Errors (WLS):

$SE(\hat{\theta}_1) = 21.468751$

$SE(\hat{\theta}_2) = 0.010611$

4.5. FORECASTS FOR NEXT 12 MONTHS ($\lambda = 0.9$)



Forecast comparison (first 6 months):

Month	OLS	WLS ($\lambda=0.9$)	Difference
1	3.2812	3.2377	-0.0435
2	3.2858	3.2400	-0.0459
3	3.2905	3.2422	-0.0483
4	3.2952	3.2445	-0.0506
5	3.2999	3.2468	-0.0530
6	3.3045	3.2491	-0.0554

4.5. COMMENT ON PREDICTIONS

Which predictions would you choose?

The WLS model with $\lambda=0.9$ emphasizes recent data points, resulting in a potentially different slope compared to OLS. Key observations:

- WLS Advantages:**
 - Captures the most recent trend more accurately
 - Better for adaptive forecasting when trend is changing
 - Recent observations have exponentially higher weights
- OLS Advantages:**
 - Uses all historical data equally
 - More stable if data generation process hasn't changed
- Choice depends on:**
 - If you believe the recent trend is more representative → WLS
 - If you believe the long-term trend is stable → OLS

For this vehicle registration data, WLS may be preferable if the market dynamics have shifted recently (e.g., EV adoption policies).

5: Recursive

```
In [12]: Dtrain.head()
```

```
Out[12]:
```

	time	year	total
0	2018-01-01 00:00:00+00:00	2018.000000	2.930483
1	2018-02-01 00:00:00+00:00	2018.083333	2.934044
2	2018-03-01 00:00:00+00:00	2018.166667	2.941422
3	2018-04-01 00:00:00+00:00	2018.250000	2.951498
4	2018-05-01 00:00:00+00:00	2018.333333	2.963093

```
In [13]: # RLS update equations up to t = 3
p = X.shape[1]
y_vec = np.asarray(y).reshape(-1)

# Initializations from assignment:
R0 = 0.1 * np.eye(p)
theta0 = np.zeros((p, 1))
```



```

R_prev = R0.copy()
theta_prev = theta0.copy()

theta_path = []

for t in range(1, 4):
    x_t = X[t - 1].reshape(-1, 1)
    y_t = float(y_vec[t - 1])

    R_t = R_prev + x_t @ x_t.T
    innovation = y_t - float((x_t.T @ theta_prev).item())
    theta_t = theta_prev + np.linalg.inv(R_t) @ x_t * innovation

    theta_path.append([t, theta_t[0, 0], theta_t[1, 0]])

    R_prev = R_t
    theta_prev = theta_t

theta_recursive_t3 = theta_prev

print("Initial values:")
print(R0)
print(theta0)
print("\nRecursive estimates up to t = 3:")
display(pd.DataFrame(theta_path, columns=["t", "theta_1", "theta_2"]))
print(f"\n $\hat{\theta}_3 = [{theta\_recursive\_t3[0,0]:.6f}, {theta\_recursive\_t3[1,0]:.6f}]$ ")

```

Initial values:

```

[[0.1 0. ]
 [0.  0.1]]
[[0.]
 [0.]]

```

Recursive estimates up to t = 3:

	t	theta_1	theta_2
0	1	7.196093e-07	0.001452
1	2	9.775061e-09	0.001453
2	3	-3.696235e-06	0.001455

$\hat{\theta}_3 = [-0.000004, 0.001455]$

Why include the closed-form expression?

- **Closed form** means a direct formula for the final estimate, without running the recursive loop step by step.
- For this RLS setup, $\hat{\theta}_N = (R_0 + X^T X)^{-1}(R_0 \hat{\theta}_0 + X^T y)$.
- We compute it here as a **check**: if recursive RLS and this formula match, the loop implementation is correct.
- With your initialization $R_0 = 0.1I$ and $\hat{\theta}_0 = 0$, this becomes $\hat{\theta}_N = (X^T X + 0.1I)^{-1} X^T y$, which is slightly regularized, so it is not exactly the same as pure OLS.

```

In [14]: # RLS estimate at t = N and comparison with OLS

p = X.shape[1]
N = X.shape[0]
y_vec = np.asarray(y).reshape(-1)

# Same initialization as requested
R0 = 0.1 * np.eye(p)
theta0 = np.zeros((p, 1))

R_prev = R0.copy()
theta_prev = theta0.copy()

for t in range(1, N + 1):
    x_t = X[t - 1].reshape(-1, 1)
    y_t = float(y_vec[t - 1])

    R_t = R_prev + x_t @ x_t.T
    innovation = y_t - float((x_t.T @ theta_prev).item())
    theta_t = theta_prev + np.linalg.inv(R_t) @ x_t * innovation

    R_prev = R_t
    theta_prev = theta_t

theta_rls_N = theta_prev.reshape(-1)
theta_ols = np.asarray(theta_hat).reshape(-1)

# Closed-form target implied by initialization (ridge-like normal equation)
theta_rls_closed = np.linalg.inv(R0 + X.T @ X) @ (R0 @ theta0).reshape(-1) + np.

diff_ols = theta_rls_N - theta_ols
diff_closed = theta_rls_N - theta_rls_closed

comparison = pd.DataFrame({
    "parameter": ["theta_1 (intercept)", "theta_2 (slope)"],
    "RLS_tN": theta_rls_N,
    "OLS": theta_ols,
    "RLS_minus_OLS": diff_ols,
    "RLS_closed_form": theta_rls_closed,
    "RLS_minus_closed": diff_closed
})

print(f"N = {N}")
print("\nRLS at t=N compared with OLS and closed-form RLS target:")
display(comparison)
print(f"Max |RLS - OLS|: {np.max(np.abs(diff_ols)).12f}")
print(f"Max |RLS - closed-form RLS|: {np.max(np.abs(diff_closed)).12e}")

```

N = 72

RLS at t=N compared with OLS and closed-form RLS target:

	parameter	RLS_tN	OLS	RLS_minus_OLS	RLS_closed_form	RLS_minus_closed
0	theta_1 (intercept)	-0.058319	-110.355428	110.297109	-0.058319	-1.064003e-12
1	theta_2 (slope)	0.001568	0.056145	-0.054577	0.001568	3.855423e-16

Max |RLS - OLS|: 110.297108664870

Max |RLS - closed-form RLS|: 1.064003052331e-12

```
In [15]: # Sensitivity of RLS_N to initialization: component-wise plots

p = X.shape[1]
N = X.shape[0]
y_vec = np.asarray(y).reshape(-1)
theta_ols = np.asarray(theta_hat).reshape(-1)

def rls_final_theta(alpha_R0, theta0_vec):
    R_prev = alpha_R0 * np.eye(p)
    theta_prev = np.asarray(theta0_vec, dtype=float).reshape(-1, 1)

    for t in range(1, N + 1):
        x_t = X[t - 1].reshape(-1, 1)
        y_t = float(y_vec[t - 1])

        R_t = R_prev + x_t @ x_t.T
        innovation = y_t - float((x_t.T @ theta_prev).item())

        try:
            gain_term = np.linalg.solve(R_t, x_t)
        except np.linalg.LinAlgError:
            gain_term = np.linalg.pinv(R_t) @ x_t

        theta_t = theta_prev + gain_term * innovation

        R_prev = R_t
        theta_prev = theta_t

    return theta_prev.reshape(-1)

# --- Plot 1: component-wise difference vs alpha (changing initial R) ---
alphas = np.logspace(-12, 8, 140)
assignment_alpha = 0.1
assignment_theta01 = 0.0
assignment_theta02 = 0.0

diff_theta1_alpha = []
diff_theta2_alpha = []
for a in alphas:
    theta_rls = rls_final_theta(a, np.array([0.0, 0.0]))
    diff_vec = theta_rls - theta_ols
    diff_theta1_alpha.append(abs(diff_vec[0]))
    diff_theta2_alpha.append(abs(diff_vec[1]))

# --- Plot 2: component-wise difference vs theta0_1 ---
# Vary theta0 as [theta0_1, 0], keeping alpha fixed.
alpha_fixed = 0.1
theta0_1_range = np.linspace(-300, 300, 300)

diff_theta1_theta01 = []
diff_theta2_theta01 = []
l2_theta01 = []

for t01 in theta0_1_range:
```

```

theta_rls = rls_final_theta(alpha_fixed, np.array([t01, 0.0]))
diff_vec = theta_rls - theta_ols
diff_theta1_theta01.append(abs(diff_vec[0]))
diff_theta2_theta01.append(abs(diff_vec[1]))
l2_theta01.append(np.linalg.norm(diff_vec, ord=2))

# --- Plot 3: component-wise difference vs theta0_2 ---
# Vary theta0 as [0, theta0_2], keeping alpha fixed.
theta0_2_range = np.linspace(-1.0, 1.0, 300)

diff_theta1_theta02 = []
diff_theta2_theta02 = []
l2_theta02 = []

for t02 in theta0_2_range:
    theta_rls = rls_final_theta(alpha_fixed, np.array([0.0, t02]))
    diff_vec = theta_rls - theta_ols
    diff_theta1_theta02.append(abs(diff_vec[0]))
    diff_theta2_theta02.append(abs(diff_vec[1]))
    l2_theta02.append(np.linalg.norm(diff_vec, ord=2))

# --- Combined figure (3 panels) ---
fig, axes = plt.subplots(1, 3, figsize=(20, 5))

# Panel 1: vary initial R
axes[0].plot(alphas, diff_theta1_alpha, label="theta_1 at time N")
axes[0].plot(alphas, diff_theta2_alpha, label="theta_2 at time N")
axes[0].axvline(assignment_alpha, color="red", linestyle="--", linewidth=1.5, label="assignment")
axes[0].set_xscale("log")
axes[0].set_yscale("log")
axes[0].set_xlabel("alpha in  $R_0 = \alpha * I$ ")
axes[0].set_ylabel("Absolute difference to OLS")
axes[0].set_title("Changing initial R ( $R_0 = \alpha * I$ )")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

# Panel 2: vary theta0_1
axes[1].plot(theta0_1_range, diff_theta1_theta01, label="theta_1 at time N")
axes[1].plot(theta0_1_range, diff_theta2_theta01, label="theta_2 at time N")
axes[1].axvline(assignment_theta01, color="red", linestyle="--", linewidth=1.5, label="assignment")
axes[1].set_yscale("log")
axes[1].set_xlabel("theta0_1 in  $\theta_0 = [\theta_0_1, 0]$ ")
axes[1].set_ylabel("Absolute difference to OLS")
axes[1].set_title(f"Changing initial theta_1")
axes[1].grid(True, alpha=0.3)
axes[1].legend()

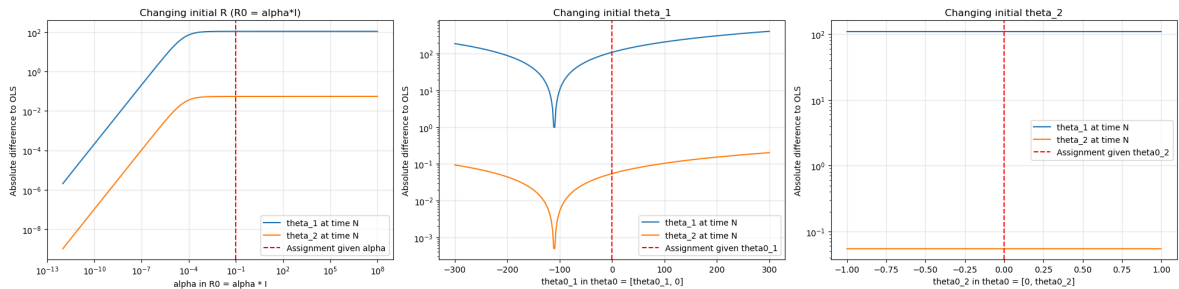
# Panel 3: vary theta0_2
axes[2].plot(theta0_2_range, diff_theta1_theta02, label="theta_1 at time N")
axes[2].plot(theta0_2_range, diff_theta2_theta02, label="theta_2 at time N")
axes[2].axvline(assignment_theta02, color="red", linestyle="--", linewidth=1.5, label="assignment")
axes[2].set_yscale("log")
axes[2].set_xlabel("theta0_2 in  $\theta_0 = [0, \theta_0_2]$ ")
axes[2].set_ylabel("Absolute difference to OLS")
axes[2].set_title(f"Changing initial theta_2")
axes[2].grid(True, alpha=0.3)
axes[2].legend()

```

```
plt.tight_layout()
plt.show()

# Report best theta0 values in scanned ranges
best_idx_01 = int(np.argmin(l2_theta01))
best_idx_02 = int(np.argmin(l2_theta02))

print("Best scanned values (for fixed alpha):")
print(f"alpha_fixed = {alpha_fixed:g}")
print(f"theta0_1* = {theta0_1_range[best_idx_01]:.6f}, min L2 = {l2_theta01[best_idx_01]:.6f}")
print(f"theta0_2* = {theta0_2_range[best_idx_02]:.6f}, min L2 = {l2_theta02[best_idx_02]:.6f}")
```



Best scanned values (for fixed alpha):
alpha_fixed = 0.1
theta0_1* = -109.364548, min L2 = 9.903836e-01
theta0_2* = 1.000000, min L2 = 1.102966e+02

Forgetting

In [16]: *# RLS with forgetting factor: compare Lambda = 0.7 vs Lambda = 0.99*

```
p = X.shape[1]
N = X.shape[0]
y_vec = np.asarray(y).reshape(-1)

def rls_with_forgetting(X, y_vec, lambda_, R0=None, theta0=None):
    """Return full parameter path for RLS with exponential forgetting."""
    n, p_local = X.shape

    if R0 is None:
        R_prev = 0.1 * np.eye(p_local)
    else:
        R_prev = np.asarray(R0, dtype=float).copy()

    if theta0 is None:
        theta_prev = np.zeros((p_local, 1))
    else:
        theta_prev = np.asarray(theta0, dtype=float).reshape(-1, 1)

    theta_hist = np.zeros((n, p_local))

    for t in range(1, n + 1):
        x_t = X[t - 1].reshape(-1, 1)
        y_t = float(y_vec[t - 1])

        # Forgetting enters here: previous information is downweighted by Lambda
        R_t = lambda_ * R_prev + x_t @ x_t.T
        innovation = y_t - float((x_t.T @ theta_prev).item())
```

```

    try:
        gain_term = np.linalg.solve(R_t, x_t)
    except np.linalg.LinAlgError:
        gain_term = np.linalg.pinv(R_t) @ x_t

    theta_t = theta_prev + gain_term * innovation

    theta_hist[t - 1, :] = theta_t.reshape(-1)
    R_prev = R_t
    theta_prev = theta_t

    return theta_hist

# Run for both forgetting factors
lambdas = [0.7, 0.99]
paths = {lam: rls_with_forgetting(X, y_vec, lam) for lam in lambdas}

# Burn-in removal for plotting (recursive estimates can be unstable initially)
burn_in = max(5, int(0.05 * N))
t_idx = np.arange(1, N + 1)
plot_idx = t_idx[burn_in:]

fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharex=True)

# Plot theta_1 path
axes[0].plot(plot_idx, paths[0.7][burn_in:, 0], label="lambda = 0.7", linewidth=2)
axes[0].plot(plot_idx, paths[0.99][burn_in:, 0], label="lambda = 0.99", linewidth=2)
axes[0].set_title(r"$\hat{\theta}_1(t)$ (intercept)")
axes[0].set_xlabel("t")
axes[0].set_ylabel(r"Estimate")
axes[0].grid(True, alpha=0.3)
axes[0].legend()

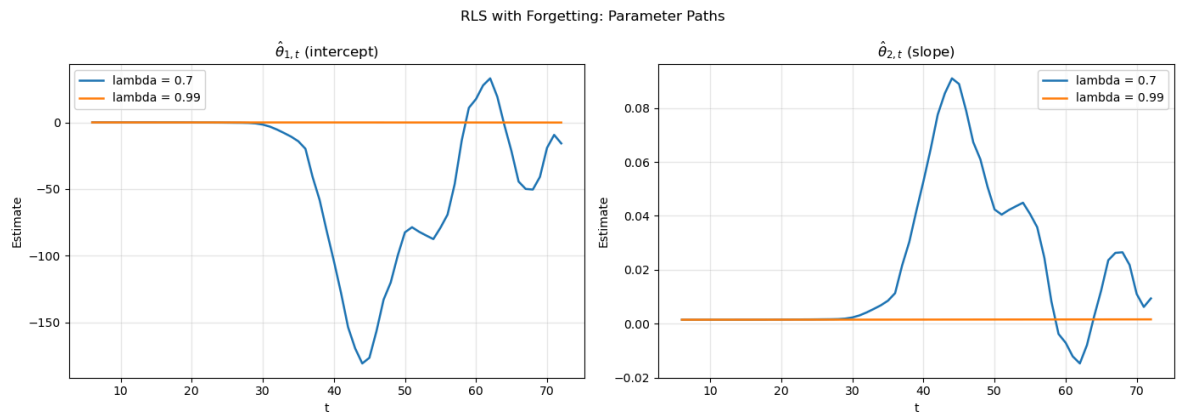
# Plot theta_2 path
axes[1].plot(plot_idx, paths[0.7][burn_in:, 1], label="lambda = 0.7", linewidth=2)
axes[1].plot(plot_idx, paths[0.99][burn_in:, 1], label="lambda = 0.99", linewidth=2)
axes[1].set_title(r"$\hat{\theta}_2(t)$ (slope)")
axes[1].set_xlabel("t")
axes[1].set_ylabel(r"Estimate")
axes[1].grid(True, alpha=0.3)
axes[1].legend()

plt.suptitle("RLS with Forgetting: Parameter Paths")
plt.tight_layout()
plt.show()

# Compact numerical comment to support plot interpretation
tail_window = max(10, int(0.1 * N))
for lam in lambdas:
    tail_std = np.std(paths[lam][-tail_window:, :], axis=0)
    print(f"lambda={lam}: tail std(theta_1)={tail_std[0]:.6e}, tail std(theta_2)={tail_std[1]:.6e}")

print("\nComment:")
print("- lambda=0.7 reacts faster to new data, so estimates typically fluctuate")
print("- lambda=0.99 gives smoother, more stable trajectories because older data")

```



$\lambda=0.7$: tail std(θ_1)= 2.175744×10^1 , tail std(θ_2)= 1.075529×10^{-2}
 $\lambda=0.99$: tail std(θ_1)= 7.002431×10^{-3} , tail std(θ_2)= 6.371509×10^{-6}

Comment:

- $\lambda=0.7$ reacts faster to new data, so estimates typically fluctuate more.
- $\lambda=0.99$ gives smoother, more stable trajectories because older data is retained longer.

5.5 Make one-step predictions

```
In [17]: # 5.5: One-step ahead predictions and residuals
# For each lambda, calculate one-step residuals:  $\hat{\varepsilon}_{t+1|t} = \hat{y}_{t+1|t} - y_{t+1}$ 

one_step_residuals = {}

for lam in lambdas:
    theta_path = paths[lam] # shape: (N, p)

    # One-step predictions ( $t=1, \dots, N-1$ ):  $\hat{y}_{t+1|t} = x_{t+1}^T \hat{\theta}_t$ 
    # theta_hat[:, 0] is theta_1 (intercept), theta_hat[:, 1] is theta_2 (slope)
    # For equation:  $y_t = \theta_1 + \theta_2 * x_t$ 
    one_step_preds = np.zeros(N - 1)

    for t in range(N - 1):
        # Predict  $y_{t+1}$  using parameter estimate at time t
        x_next = X[t + 1] # [1,  $x_{t+1}$ ]
        one_step_preds[t] = x_next @ theta_path[t]

    # Residuals:  $\hat{\varepsilon}_{t+1|t} = \hat{y}_{t+1|t} - y_{t+1}$ 
    #  $y_{t+1}$  corresponds to  $y_{\text{vec}}[1:]$  when predictions are  $y_{\text{vec}}[:-1]$ 
    residuals_1step = one_step_preds - y_vec[1:]
    one_step_residuals[lam] = residuals_1step
```